

NAME: SHREYA GHOLASE
PRN NO: 23070521145
SEC: B

Practical 5 Part II

What is a Join?

A **JOIN** combines records from two or more tables using a related column.

Types of Joins:

1. **INNER JOIN** → Returns only matching records.
2. **LEFT JOIN** → Returns all records from the left table and matching records from the right table.
3. **RIGHT JOIN** → Returns all records from the right table and matching records from the left table.
4. **FULL OUTER JOIN** → Returns all records from both tables (not available in MySQL).
5. **CROSS JOIN** → Returns the Cartesian product of both tables.
6. **SELF JOIN** → Joins a table to itself.

1. Customer Table

Column	Data Type	Constraints
customer_id	NUMBER (PK)	PRIMARY KEY, AUTO-INCREMENT

name	VARCHAR2(100)	NOT NULL
email	VARCHAR2(100)	UNIQUE
phone	VARCHAR2(15)	NOT NULL
address	VARCHAR2(255)	NULLABLE

2. Product Table

Column	Data Type	Constraints
product_id	NUMBER (PK)	PRIMARY KEY
name	VARCHAR2(100)	NOT NULL
category	VARCHAR2(50)	NOT NULL
price	DECIMAL(10,2)	NOT NULL
stock_quantity	INT	NOT NULL

3. Order_Details Table

Column	Data Type	Constraints
order_id	NUMBER (PK)	PRIMARY KEY
customer_id	NUMBER (FK)	FOREIGN KEY REFERENCES Customer(customer_id)
order_date	DATE	NOT NULL
total_amount	DECIMAL(10,2)	NOT NULL

4. Order_Item Table

Column	Data Type	Constraints
--------	-----------	-------------

order_id	NUMBER (FK)	FOREIGN KEY REFERENCES Order_Details(order_id)
product_id	NUMBER (FK)	FOREIGN KEY REFERENCES Product(product_id)
quantity	INT	NOT NULL
subtotal	DECIMAL(10,2)	NOT NULL

5. Employee Table

Column	Data Type	Constraints
--------	-----------	-------------

employee_id	NUMBER (PK)	PRIMARY KEY
name	VARCHAR2(100)	NOT NULL
role	VARCHAR2(50)	NOT NULL
salary	DECIMAL(10,2)	NOT NULL
hire_date	DATE	NOT NULL

Examples of Joins

INNER JOIN: Get order details with customer names

```
SELECT o.order_id, c.name, o.order_date, o.total_amount
FROM Order_Details o
INNER JOIN Customer c ON o.customer_id = c.customer_id;
```


NAME	
QUANTITY	
Milk	2
Bread	3
Eggs	1
NAME	
QUANTITY	
Chicken	2
Apples	5
Orange Juice	2
6 rows selected.	

LEFT JOIN: Get all customers and their orders (including those who never ordered)

```
SELECT c.name, o.order_id, o.total_amount
FROM Customer c
LEFT JOIN Order_Details o ON c.customer_id = o.customer_id;
```

NAME		
ORDER_ID	TOTAL_AMOUNT	
Alice Johnson	101	10.5
Bob Smith	102	55.2
Charlie Brown	103	40.8
NAME		
ORDER_ID	TOTAL_AMOUNT	
David Miller	104	30
Emily Davis	105	25.5

LEFT JOIN: Retrieve all products and their order details (including those not ordered yet)

```
SELECT p.name, oi.quantity
FROM Product p
LEFT JOIN Order_Item oi ON p.product_id = oi.product_id;
```

NAME	

	QUANTITY

Milk	2
Bread	3
Eggs	1
NAME	

	QUANTITY

Chicken	2
Apples	5
Orange Juice	2
6 rows selected.	

RIGHT JOIN: Get all orders with or without employee assigned

```
SELECT o.order_id, e.name AS employee_name
FROM Order_Details o
RIGHT JOIN Employee e ON o.customer_id = e.employee_id;
```

ORDER_ID
101
Michael Scott
102
Jim Halpert
103
Pam Beesly
104
Dwight Schrute
105
Kevin Malone

RIGHT JOIN: Retrieve employees who processed orders

```
SELECT e.name, o.order_id
FROM Employee e
RIGHT JOIN Order_Details o ON e.employee_id = o.customer_id;
```


NAME	ORDER_ID
Michael Scott	101
Jim Halpert	102
Pam Beesly	103
Dwight Schrute	104
Kevin Malone	105

FULL OUTER JOIN: Get all customers and orders (Oracle SQL only)

```
SELECT c.name, o.order_id, o.total_amount
FROM Customer c
FULL OUTER JOIN Order_Details o ON c.customer_id =
o.customer_id;
```

NAME		

ORDER_ID	TOTAL_AMOUNT	

Alice Johnson	101	10.5
Bob Smith	102	55.2
Charlie Brown	103	40.8
NAME		

ORDER_ID	TOTAL_AMOUNT	

David Miller	104	30
Emily Davis	105	25.5

CROSS JOIN: Show all possible employee-product assignments

```
SELECT e.name AS employee, p.name AS product
FROM Employee e
CROSS JOIN Product p;
```

EMPLOYEE

PRODUCT

Michael Scott
Milk

Michael Scott
Bread

Michael Scott
Eggs

EMPLOYEE

PRODUCT

Michael Scott
Chicken

Michael Scott
Apples

Michael Scott
Orange Juice

EMPLOYEE

SELF JOIN: Find employees earning more than their colleagues `SELECT
e1.name AS Employee, e2.name AS Colleague, e1.salary FROM
Employee e1
JOIN Employee e2 ON e1.salary > e2.salary;`

EMPLOYEE

COLLEAGUE

SALARY

Michael Scott
Jim Halpert
75000
Michael Scott
Pam Beesly
75000
EMPLOYEE

COLLEAGUE

SALARY

Michael Scott
Dwight Schrute
75000

SELF JOIN: Find employees working under the same manager

```
SELECT e1.name AS Employee, e2.name AS Manager
FROM Employee e1
JOIN Employee e2 ON e1.role = 'Cashier' AND e2.role =
'Manager';
```

EMPLOYEE

MANAGER

Jim Halpert
Michael Scott

Kevin Malone
Michael Scott

Joins Tasks

1. Retrieve **customer names** along with their orders.

```
SQL> SELECT C.name AS Customer_Name, O.order_id, O.order_date  
2 FROM CUSTOMER C  
3 JOIN ORDER_DETAILS O ON C.customer_id = O.customer_id;
```

CUSTOMER_NAME

	ORDER_ID	ORDER_DAT
Alice Johnson	101	10-JAN-24

Bob Smith	102	12-JAN-24
-----------	-----	-----------

Charlie Brown	103	01-DEC-23
---------------	-----	-----------

CUSTOMER_NAME

	ORDER_ID	ORDER_DAT
David Miller	104	05-NOV-23

Emily Davis	105	10-FEB-24
-------------	-----	-----------

2. Show **product names** and their **order quantities**.

```
SQL> SELECT P.name AS Product_Name, OI.quantity
  2  FROM PRODUCT P
  3  JOIN ORDER_ITEM OI ON P.product_id = OI.product_id;
```

PRODUCT_NAME

QUANTITY

Milk

2

Bread

3

Eggs

1

PRODUCT_NAME

QUANTITY

Chicken

2

Apples

5

Orange Juice

3. List all customers and their orders (**including those who never ordered**).

```
SQL> SELECT C.name AS Customer_Name, O.order_id, O.order_date, O.total_amount  
2 FROM CUSTOMER C  
3 LEFT JOIN ORDER_DETAILS O ON C.customer_id = O.customer_id;
```

CUSTOMER_NAME

	ORDER_ID	ORDER_DAT	TOTAL_AMOUNT
Alice Johnson	101	10-JAN-24	10.5
Bob Smith	102	12-JAN-24	55.2
Charlie Brown	103	01-DEC-23	40.8

CUSTOMER_NAME

	ORDER_ID	ORDER_DAT	TOTAL_AMOUNT
David Miller	104	05-NOV-23	30
Emily Davis	105	10-FEB-24	25.5

4. Retrieve all products and their order details (including those not ordered yet).


```

SQL> SELECT P.name AS Product_Name, OI.order_id, OI.quantity, OI.subtotal
2  FROM PRODUCT P
3  LEFT JOIN ORDER_ITEM OI ON P.product_id = OI.product_id;

```

PRODUCT_NAME	ORDER_ID	QUANTITY	SUBTOTAL
Milk	101	2	5
Bread	101	3	5.4
Eggs	102	1	3.2

PRODUCT_NAME	ORDER_ID	QUANTITY	SUBTOTAL
Chicken	102	2	15
Apples	103	5	6
Orange Juice			

5. Find employees who have **processed orders**.

```
SQL> SELECT DISTINCT E.name AS Employee_Name, E.role  
2 FROM EMPLOYEE E  
3 JOIN ORDER_DETAILS O ON E.employee_id = O.employee_id;
```

EMPLOYEE_NAME

ROLE

Michael Scott
Manager

Jim Halpert
Cashier

Pam Beesly
Sales Associate

EMPLOYEE_NAME

ROLE

Dwight Schrute
Supervisor

Kevin Malone
Cashier